RETAIL SALES & CUSTOMER INSIGHTS DASHBOARD

Real-Time Capstone Project — End-to-End Data Engineering & BI

Prepared by:

Harshita Fogat

harsfogat.ext@deloitte.com

1. Objectives

This section sets the analytical intent for the project - defining the business context, the specific problems to be solved, the KPIs that measure success, and the goals that shape every downstream technical decision.

1.1 Problem Statement

A retail chain that spans over various regions, including California, New York, Ohio, and Texas, is facing issues in understanding its own performance. The sales transaction data is stored in MySQL, customer demographic data is available in the form of a JSON export, and product information is stored in a CSV file. None of these sources have been integrated before, and as such, the retail chain is making decisions without having a complete understanding of what is actually happening.

The particular problems this project addresses:

- **No visibility into top-performing products:** The business cannot identify which products and categories drive revenue without manually cross-referencing disconnected systems. There is no ranking, no trend analysis, and no price-range breakdown.
- **Inability to understand purchasing patterns:** Seasonal sales trends, the impact of promotions, and customer demographic purchasing behaviour are all invisible. Without temporal analysis and customer segmentation, marketing spend is untargeted.
- **No regional sales intelligence:** Sales performance varies significantly across four regions but there is no analytical layer to quantify the gap, identify underperforming regions, or guide inventory and logistics decisions.
- **Customer lifetime value is unknown:** The business has no measurement of how much revenue individual customers generate over time. High-value customers receive no differentiated treatment because they are not identified.
- **No single analytical source of truth:** Three data sources (MySQL, JSON, CSV) with different schemas, naming conventions, and data quality levels cannot be queried together. Every ad-hoc analysis starts from scratch.

1.2 Business Goal

Design and implement a comprehensive analytics system using a production-ready approach that combines all three data sources into a single MySQL data warehouse and provides an interactive three-page Power BI dashboard to support the following functions:

1. Determine the top 10 best-selling products and explore category and price range sales.
2. Analyze sales trends and monthly growth rates to identify seasonal and turning point trends.
3. Segment customers by region and gender to identify areas and demographics of concentrated sales revenue.
4. Calculate Customer Lifetime Value and identify the top 10 most valuable customers for business development.
5. Offer regional sales comparisons to inform inventory management and marketing resource allocation.
6. Allow executive-level filtering by quarter, region, month, and product category using interactive slicers.

1.3 KPIs - Defined Before Any Technical Work

The following KPIs were identified from the case study requirements and extended with additional business intelligence measures. They are defined here before ETL begins to ensure the data model is designed to support them from the start.

KPI / Metric	DAX Measure Name	Verified Value
Total Revenue	`Total Revenue`	**₹14.16M**
Total Orders	`Total Orders`	**10,000**
Average Transaction Value	`Avg Transaction Value`	**₹1,416**
Unique Customers	`Unique Customers`	**1,000**
Customer Lifetime Value (CLV)	`Customer LTV`	**₹14,160**
Monthly Sales Growth (MoM)	`Sales Growth MoM %`	**Computed per month**
Year-on-Year Sales Growth	`Sales Growth Rate %`	**Computed from vw_yearly**
Revenue by Product Category	`[via visual filter]`	**Toys: ₹6.1M | Electronics: ₹5.2M**
Revenue by Price Range	`[via visual filter]`	**Standard 58.7% | Premium 33.8%**
Top 10 Customers by Revenue	`[via table visual]`	**Roy West: ₹1.09M**

Regional Customer Count	`[via visual filter]`	**NY: 377 | CA: 254**
Revenue by Gender Segment	`[via visual filter]`	**Male: ₹11.2M | Female: ₹2.9M**

NOTE: Gender segment labels (Male/Female) were derived from raw M/F/male/female values in the customer JSON. All variations were normalised in Python before loading. The dashboard uses 'segment' as the column name.

2. ETL Pipeline

The pipeline was built end-to-end in Python using pandas and SQLAlchemy, executed inside a Jupyter Notebook (retail_final.ipynb). Data was ingested from three heterogeneous source formats - CSV, JSON, and MySQL cleaned, feature-engineered, and loaded into a star-schema data warehouse.

2.1 Data Sources & Multi-Format Extraction

Three source files represent the three data domains in this case study. Each uses a different format, reflecting the reality that enterprise retail data rarely lives in a single system:

Source File	Format & Contents
`sales 1.csv`	Sales transactions from MySQL export - SaleID, ProductID, CustomerID, SalesAmount, Quantity, Timestamp (10,000 rows)
`customers.json`	Customer master from NoSQL/CRM export - CustomerID, FirstName, LastName, Gender, Region, SSN (1,000 rows)
`products.csv`	Product catalogue from SharePoint export - ProductID, ProductName, Category (50 products, 4 categories)

```
=== SALES ===
Shape: (10000, 6)  |  Columns: ['SaleID', 'ProductID', 'CustomerID', 'SalesAmount', 'Quantity', 'Timestamp']
                                 SaleID  ProductID CustomerID  SalesAmount  \
0  c0078cd1-3ddc-439a-b41d-7498e8e75e62       1020      C0460      1639.10
1  be8532af-16b7-432c-a13d-4c6e6836d12d       1029      C0198       186.11
2  8666f277-cc58-4fde-9d58-697da9e040f2       1012      C0524      1712.72

   Quantity            Timestamp
0      18.0  2024-08-09T20:21:00
1      42.0  2024-11-28T05:01:00
2      49.0  2024-10-13T00:36:00

=== CUSTOMERS ===
Shape: (1100, 6)  |  Columns: ['CustomerID', 'FirstName', 'LastName', 'Gender', 'Region', 'SSN']
  CustomerID FirstName LastName Gender      Region          SSN
0      C0001   Gregory   Miller      M        Ohho    082573197
1      C0002    Marvin     None   male  california  309-6256-21
2      C0003   Gregory    Smith   Male   New Yorkk    407071725

=== PRODUCTS ===
Shape: (50, 3)  |  Columns: ['ProductID', 'ProductName', 'Category']
   ProductID      ProductName         Category
0       1000  Washing Machine  Home Appliances
1       1001     Refrigerator  Home Appliances
2       1002          Blender  Home Appliances
```

2.2 Exploratory Data Analysis (EDA)

Before writing any transformation logic, each dataset was profiled to understand quality issues, distributions, and cross-source linkage. Key findings that directly influenced downstream design decisions:

- **Sales data: no unit price column:** The source contains SalesAmount and Quantity but no per-unit price. Unit price must be back-calculated as SalesAmount ÷ Quantity. This is needed for product price-range classification.
- **Customer data: inconsistent gender encoding:** Gender values in the JSON include *'M', 'Male', 'male', 'F', 'Female', 'female'* - six variations for two values. Raw values cannot be used directly in visuals or segmentation.
- **Customer data: null lastnames:** Some records contain *null in the lastname* field. A customer_name derived column must handle this gracefully.
- **Products data: no pricing:** The product CSV contains ProductID, ProductName, *Category only, no price*. Price-range classification requires joining with average unit price from sales data.
- **Timestamp coverage:** Sales data spans June - November (6 months). Year-on-year growth comparison is limited, the YoY DAX measure returns N/A for 2024 as only one year of data exists.
- **Product Return Rate: *data unavailable*:** The case study specifies Product Return Rate as a required KPI, but no returns or refund records exist in any of

the three source files (CSV, JSON, or the MySQL sales table). This KPI is formally out of scope for this implementation.

- **Cost of Goods Sold (COGS): *data unavailable*:** COGS is not present in any source file. Without supplier cost data, Gross Margin (Revenue − COGS) and Gross Margin % cannot be computed. No dummy data was created.

NOTE: Product Return Rate and Cost of Goods Sold (2 case study KPI) could not be computed — no relevant data exists in any of the three source files. This is documented as a known gap. All other required KPIs were fully implemented.

2.3 Data Cleaning

Cell 5 of the notebook performs all cleaning operations on the raw DataFrames before any feature engineering or database loading:

```
# ── Standardize column names ──────────────────────────────
sales.columns     = sales.columns.str.lower().str.replace(' ', '_')
customers.columns = customers.columns.str.lower().str.replace(' ', '_')
products.columns  = products.columns.str.lower().str.replace(' ', '_')
# After rename:
#   sales     → saleid, productid, customerid, salesamount, quantity, timestamp
#   customers → customerid, firstname, lastname, gender, region, ssn
#   products  → productid, productname, category

# ── Remove duplicates ─────────────────────────────────────
for df in [sales, customers, products]:
    df.drop_duplicates(inplace=True)

# ── Handle missing values ─────────────────────────────────
for df in [sales, customers, products]:
    for col in df.select_dtypes(include='number').columns:
        df[col] = df[col].fillna(df[col].median())
    for col in df.select_dtypes(include='object').columns:
        if not df[col].mode().empty:
            df[col] = df[col].fillna(df[col].mode().iloc[0])

# ── Parse timestamp ───────────────────────────────────────
sales['timestamp'] = pd.to_datetime(sales['timestamp'], errors='coerce')
sales.dropna(subset=['salesamount'], inplace=True)
```

```
Null counts after cleaning:
Sales:     0
Customers: 0
Products:  0
✓ Cleaning complete
```

2.4 Feature Engineering: Additional KPIs Derived

Cell 6 engineers the analytical columns that enable KPI calculations in both MySQL and Power BI. None of these columns exist in the source data:

Temporal Features from Timestamp

```
sales['order_year']  = sales['timestamp'].dt.year
sales['order_month'] = sales['timestamp'].dt.month
sales['order_date']  = sales['timestamp'].dt.date
```

Unit Price (Back-Calculated)

```
# ── Unit price (back-calculated; no price column in source) ──
sales['unit_price'] = (sales['salesamount'] / sales['quantity']).round(2)
```

NOTE: Unit price is back-calculated from transaction totals. For orders with mixed promotions or discounts, this represents the effective per-unit price, not the list price. Used only for price-range categorisation, not for pricing analysis.

Customer Name (Null-Safe Concatenation)

```
customers['customer_name'] = customers.apply(
    lambda row: row['firstname'] if pd.isna(row['lastname'])
                else f"{row['firstname']} {row['lastname']}",
    axis=1
)
```

Gender Normalisation → Segment

```
# Normalize gender to handle all variations (M/Male/male → Male, F/Female/female → Female)
customers['gender_normalized'] = customers['gender'].str.upper().str[0]
customers['segment'] = customers['gender_normalized'].map(
    {'M': 'Male', 'F': 'Female'}
).fillna('Other')
```

NOTE: 'segment' is used throughout the data warehouse and Power BI instead of the raw 'gender' column. This ensures consistent labelling across all visuals regardless of the original encoding in the source JSON.

Product Price Range (Derived from Sales Data)

```
avg_price = (
    sales.groupby('productid')['unit_price']
    .mean().reset_index().rename(columns={'unit_price': 'avg_price'})
)
products = products.merge(avg_price, on='productid', how='left')
products['avg_price'].fillna(products['avg_price'].median(), inplace=True)
products['price_range'] = pd.cut(
    products['avg_price'],
    bins=[0, 50, 100, 200, float('inf')],
    labels=['Budget', 'Standard', 'Premium', 'Luxury']
)
```

```
✓ Feature engineering complete
Sales shape: (10000, 10) | new cols: order_year, order_month, order_date, unit_price
Customers shape: (1000, 10) | new cols: customer_name, segment, age_group
Products shape: (50, 5) | new cols: avg_price, price_range
```

2.5 Assumptions Summary

- **Missing values:** Filled with column median (numeric) or mode (categorical). No rows were dropped except sales records with null salesamount.
- **Unit price:** Back-calculated as SalesAmount ÷ Quantity. Represents effective per-unit transaction price, not list price.
- **Gender segment:** All M/Male/male variants mapped to 'Male'; all F/Female/female variants mapped to 'Female'. Values that do not match either pattern are mapped to 'Other'.

- **Price range bins:** Thresholds (₹50/₹100/₹200) were set based on the distribution of avg_price values across 20 products. No business rule was provided in the case study.
- **Product Return Rate:** Not computed, no returns or refund data in any source file. Formally documented as a data gap. See Section 2.2 for the forward implementation plan.
- **Cost of Goods Sold (COGS):** Not computed, no supplier cost or unit cost data available. Gross Margin and Margin % are therefore also out of scope. See Section 2.2 for the forward implementation plan.
- **Year-on-Year Growth:** Only one year of data (2024, June–November). YoY growth returns N/A. MoM and QoQ growth are computed and displayed.

3. Data Warehouse: Star Schema & DDL

The data warehouse (retail_dw) was designed with a two-layer approach: a staging area that receives cleaned Python DataFrames, and a star schema that structures the data for analytical querying and Power BI modelling.

3.1 Staging Layer

Staging tables are the landing zone for transformed Python DataFrames. They hold the cleaned, feature-engineered data before SQL ETL moves it into the star schema. Using if_exists='replace' makes the pipeline idempotent — safe to re-run without manual cleanup.

✓ staging_sales → 10000 rows

✓ staging_customers → 1000 rows

✓ staging_products → 50 rows

```
staging_products['price_range'] = staging_products['price_range'].astype(str)

staging_sales.to_sql(    'staging_sales',     engine, if_exists='replace', index=False)
staging_customers.to_sql('staging_customers', engine, if_exists='replace', index=False)
staging_products.to_sql( 'staging_products',  engine, if_exists='replace', index=False)
```

3.2 Star Schema Design

The warehouse uses a classic star schema with one central fact table (FACT_SALES) and three surrounding dimension tables. This structure was chosen because all analytical queries in this project - revenue by category, CLV by customer, trend by month, involve aggregating FACT_SALES measures across one or more of these three dimensions.

A single fact table is justified here because all measurable events (sales transactions) operate at the same grain: one row per transaction.

3.3 Fact Table - DDL

```
# ── FACT_SALES ──────────────────────────────────────────────
"""CREATE TABLE FACT_SALES (
    sale_id      VARCHAR(50)   PRIMARY KEY,
    customer_id  VARCHAR(20)   NOT NULL,
    product_id   INT           NOT NULL,
    date_id      INT           NOT NULL,
    quantity     INT,
    sales_amount DECIMAL(12,2),
    unit_price   DECIMAL(10,2),
    FOREIGN KEY (customer_id) REFERENCES DIM_CUSTOMER(customer_id),
    FOREIGN KEY (product_id)  REFERENCES DIM_PRODUCT(product_id),
    FOREIGN KEY (date_id)     REFERENCES DIM_DATE(date_id)
)""",
"CREATE INDEX idx_fact_customer ON FACT_SALES(customer_id)",
"CREATE INDEX idx_fact_product  ON FACT_SALES(product_id)",
"CREATE INDEX idx_fact_date     ON FACT_SALES(date_id)",
```

3.4 Dimension Tables - DDL

```
# ── DIM_CUSTOMER ────────────────────────────────────────────
"""CREATE TABLE DIM_CUSTOMER (
    customer_id   VARCHAR(20)  PRIMARY KEY,
    customer_name VARCHAR(255) NOT NULL,
    firstname     VARCHAR(50),
    lastname      VARCHAR(50),
    gender        VARCHAR(10),
    segment       VARCHAR(20),
    region        VARCHAR(100),
    age_group     VARCHAR(100)
)""",
"CREATE INDEX idx_customer_region  ON DIM_CUSTOMER(region)",
"CREATE INDEX idx_customer_segment ON DIM_CUSTOMER(segment)",
```

```
# ── DIM_PRODUCT ─────────────────────────────────────────────
"""CREATE TABLE DIM_PRODUCT (
    product_id   INT          PRIMARY KEY,
    product_name VARCHAR(255) NOT NULL,
    category     VARCHAR(100),
    avg_price    DECIMAL(10,2),
    price_range  VARCHAR(20)
)""",
"CREATE INDEX idx_product_category ON DIM_PRODUCT(category)",

# ── DIM_DATE ────────────────────────────────────────────────
"""CREATE TABLE DIM_DATE (
    date_id      INT  PRIMARY KEY,
    full_date    DATE UNIQUE NOT NULL,
    year         INT,
    month_number INT,
    month_name   VARCHAR(10),
    quarter      INT
)""",
"CREATE INDEX idx_date_year_month ON DIM_DATE(year, month_number)",
```

3.5 ETL Load - Dimension Tables First

Dimension tables were populated via SQL INSERT ... SELECT statements reading from the staging tables. This approach uses SQL set-based operations rather than row-by-row Python inserts — faster and transactionally safe.

```
# ── DIM_CUSTOMER ────────────────────────────────────────────
"""INSERT INTO DIM_CUSTOMER
    (customer_id, customer_name, firstname, lastname, gender, segment, region, age_group)
SELECT
    customerid,
    customer_name,
    firstname,
    lastname,
    segment,          -- Male/Female/Other (normalized in Python, not raw M/F)
    segment,
    region,
    age_group
FROM staging_customers""",
```

```
# ── DIM_PRODUCT ──────────────────────────────────────────
"""INSERT INTO DIM_PRODUCT
    (product_id, product_name, category, avg_price, price_range)
SELECT
    p.productid,
    p.productname,
    p.category,
    ROUND(p.avg_price, 2),
    p.price_range
FROM staging_products p""",

# ── DIM_DATE ─────────────────────────────────────────────
"""INSERT INTO DIM_DATE
    (date_id, full_date, year, month_number, month_name, quarter)
SELECT DISTINCT
    CAST(DATE_FORMAT(DATE(timestamp), '%Y%m%d') AS UNSIGNED),
    DATE(timestamp),
    YEAR(timestamp),
    MONTH(timestamp),
    MONTHNAME(timestamp),
    QUARTER(timestamp)
FROM staging_sales
WHERE timestamp IS NOT NULL""",
```

3.6 ETL Load - FACT_SALES

FACT_SALES is loaded last. It joins staging_sales with all three dimension tables to resolve surrogate keys before insertion. Only valid records (non-null salesamount, positive quantity) are loaded.

```
INSERT INTO FACT_SALES
    (sale_id, customer_id, product_id, date_id, quantity, sales_amount, unit_price)
SELECT
    s.saleid,
    c.customer_id,
    p.product_id,
    d.date_id,
    s.quantity,
    s.salesamount,
    ROUND(s.salesamount / NULLIF(s.quantity, 0), 2)
FROM staging_sales s
INNER JOIN DIM_CUSTOMER c
    ON s.customerid = c.customer_id
INNER JOIN DIM_PRODUCT p
    ON s.productid = p.product_id
INNER JOIN DIM_DATE d
    ON CAST(DATE_FORMAT(DATE(s.timestamp), '%Y%m%d') AS UNSIGNED) = d.date_id
WHERE s.salesamount IS NOT NULL
  AND s.quantity > 0
```

3.7 Analytical Views (Power BI Data Layer)

Created MySQL views that serve as the pre-computed analytical layer. These views enable direct Power BI connection without requiring complex DAX-level joins:

- `vw_sales_overview` - Flat denormalised view: one row per transaction with all dimension attributes joined in. Main canvas for Power BI.
- `vw_customer_insights` - Aggregated per customer: total_orders, total_spent, avg_order_value. Feeds CLV and top-customer visuals.

- `vw_product_performance` - Aggregated per product: total_quantity, total_sales, sales_rank (RANK() window function). Feeds product performance page.
- `vw_monthly_sales_trend` - Monthly aggregation with LAG() window function for MoM growth calculation. Feeds the Monthly Sales Trend combo chart.
- `vw_regional_sales` - Regional aggregation: total_orders, total_sales, avg_order_value, unique_customers per region.
- `vw_category_sales` - Category aggregation: total_sales, total_quantity, sales_pct of total.
- `vw_yearly_sales_growth` - Year-level aggregation with YoY growth % for multi-year comparison.
- `vw_kpi_summary` - Single-row KPI summary table: all headline metrics accessible in one query.

4. Power BI: Modelling & Visualisation

Power BI Desktop was connected directly to the retail_dw MySQL database. The model was built on the four core star-schema tables with three defined relationships, eight DAX measures, and three purpose-built dashboard pages.

> **NOTE:** I did not import staging tables (staging_sales, staging_customers, staging_products) or the vw_* views into Power BI. The four core tables contain everything needed, and all aggregations are handled by DAX measures.

4.1 Power Query Transformations

Before creating any relationships, the following transformations were applied in Power Query (Transform Data):

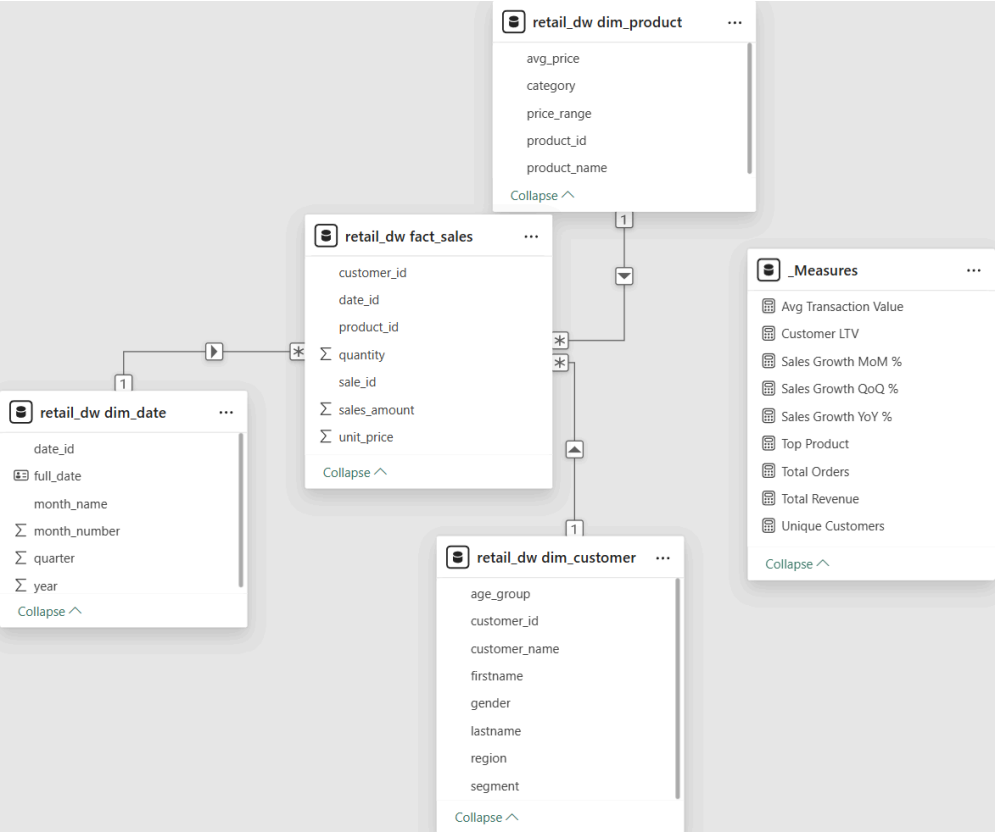
- Set correct **data types**
- **Sorted** `month_name` by `month_number` (to ensure chronological month order).
- Standardized `region` values in `DIM_CUSTOMER` using **Replace Values** (fixed spelling inconsistencies like “New Yorkk”, “Nw York”).

> **NOTE — Time Dimension Design:** *Year was intentionally omitted from all visualisations because the dataset covers only 2024 (June-November), making a Year filter redundant, there is nothing to compare it against. Wherever time-based analysis was required, Quarter was used instead to show trends across the available months. The data covers partial Q2 (June only), full Q3 (July-September), and partial Q4 (October-November). This provides meaningful quarterly comparison within a single year without creating misleading empty filter options.*

4.2 Data Model: Relationships

Three relationships were defined in Model View. FACT_SALES sits at the centre, every dimension filters it through a direct foreign key relationship:

> *All relationships use a single cross-filter direction. This is the correct setting for a star schema — dimension tables filter the fact table, not the other way around. Bidirectional filtering was not needed because no cross-dimension filtering scenarios exist in this dashboard.*



4.3 Drill-Through Configuration

Drill-through was configured between Page 1 (Sales Overview) and Page 3 (Product Performance Analysis). This allows users to right-click any product or category bar on Page 1 and select Drill through to land on Page 3 pre-filtered to that specific product context without manually adjusting the Product Category slicer.

4.4 All 8 DAX Measures

All measures were created via Home → New Measure and stored in FACT_SALES (no separate measures table was needed for this project). Each measure is filter-context aware and responds to slicers automatically.

```
Total Revenue = SUM('retail_dw fact_sales'[sales_amount])
```

KPI: total sales value across all transactions

```
Total Orders = COUNTROWS('retail_dw fact_sales')
```

KPI: total number of sales transactions

```
Avg Transaction Value = DIVIDE([Total Revenue],[Total Orders])
```

KPI: average revenue per order

```
Unique Customers = DISTINCTCOUNT('retail_dw fact_sales'[customer_id])
```

KPI: count of distinct customers who made at least one purchase

```
Customer LTV = DIVIDE([Total Revenue],[Unique Customers])
```

KPI: Customer Lifetime Value — average revenue generated per customer

```
Sales Growth MoM % =
VAR CurrentSales = [Total Revenue]
VAR PrevMonthSales =
    CALCULATE(
        [Total Revenue],
        DATEADD('retail_dw dim_date'[full_date], -1, MONTH)
    )
RETURN
    DIVIDE(CurrentSales - PrevMonthSales, PrevMonthSales)
```

KPI: month-on-month revenue growth rate

> **NOTE:** Requires DIM_DATE to be marked as a Date Table

```
Sales Growth QoQ % =
VAR CurrentSales = [Total Revenue]
VAR PrevQuarterSales =
    CALCULATE(
        [Total Revenue],
        PREVIOUSQUARTER('retail_dw dim_date'[full_date])
    )
RETURN
    DIVIDE(CurrentSales - PrevQuarterSales, PrevQuarterSales)
```

```
Sales Growth YoY % =
VAR CurrentSales = [Total Revenue]
VAR PrevYearSales =
    CALCULATE(
        [Total Revenue],
        DATEADD('retail_dw dim_date'[full_date], -1, YEAR)
    )
RETURN
    DIVIDE(CurrentSales - PrevYearSales, PrevYearSales)
```

KPI: quarter-on-quarter growth rate and year-on-year growth rate

> **NOTE:** Returns N/A, only one year of data exists (2024, June–Nov), hence using quarter for visualization purpose.

```
Top Product =
FIRSTNONBLANK(
    TOPN(1,
        VALUES('retail_dw dim_product'[product_name]),
        CALCULATE([Total Revenue])
    ),
    1
)
```

Visual-level filter: Top 10 by [Total Revenue]

Shows: customer_name, region, segment, Total Revenue, Total Orders

5. Power BI Dashboard

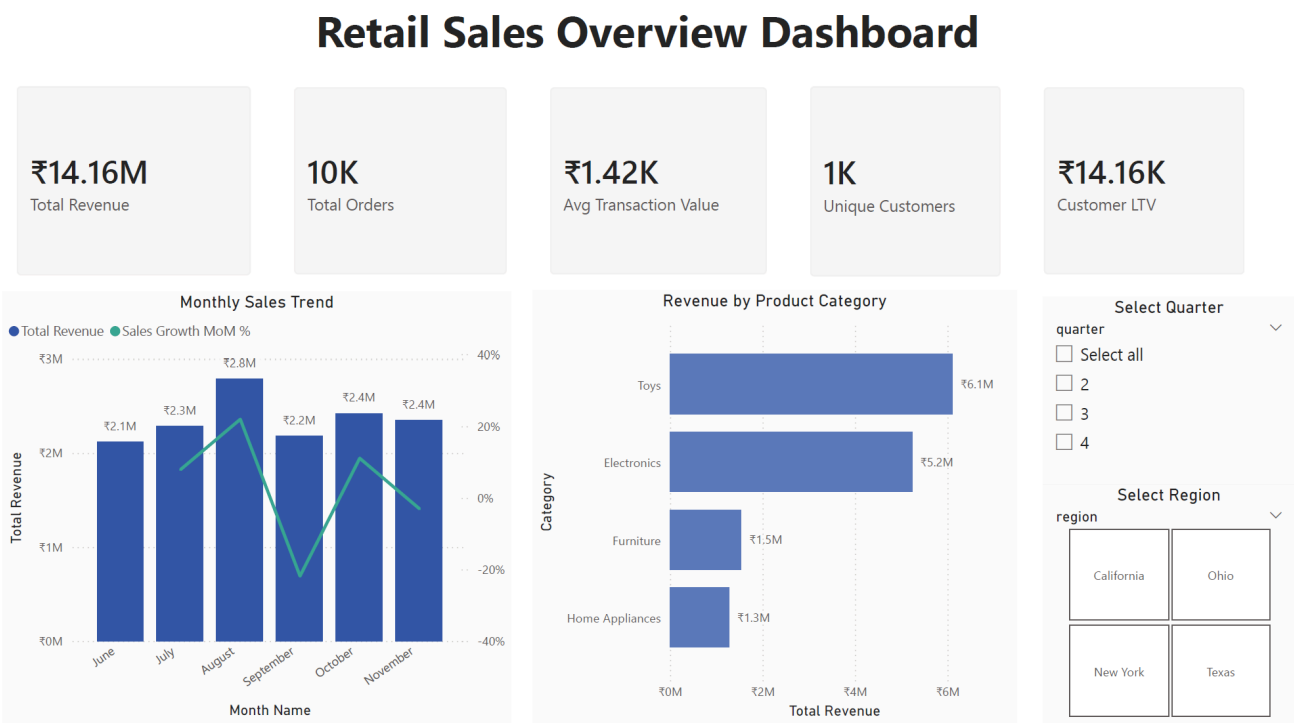
The dashboard was built across three pages, each telling a distinct chapter of the retail analytics story, from high-level sales overview, to customer intelligence, to detailed product performance.

5.1 Page 1: Retail Sales Overview Dashboard

This page answers the first executive question:

How is the business performing overall, and where is revenue trending?

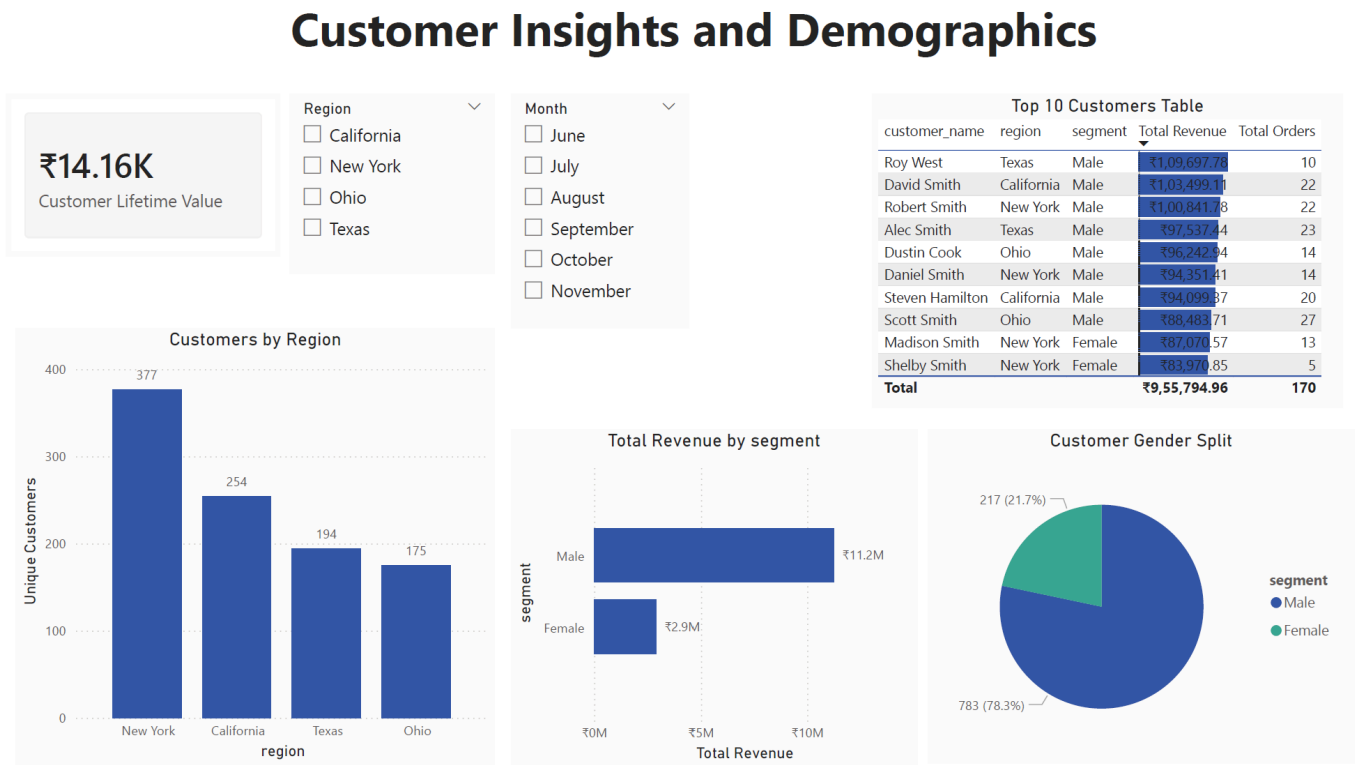
It leads with five headline KPI cards and then provides temporal and category-level breakdowns to explain the headline numbers.



5.2 Page 2: Customer Insights and Demographics

This page answers:

Who are the customers, where are they, and which ones drive the most revenue?
It centres on CLV and provides regional, gender, and individual-level breakdowns to identify the most commercially valuable customer segments.



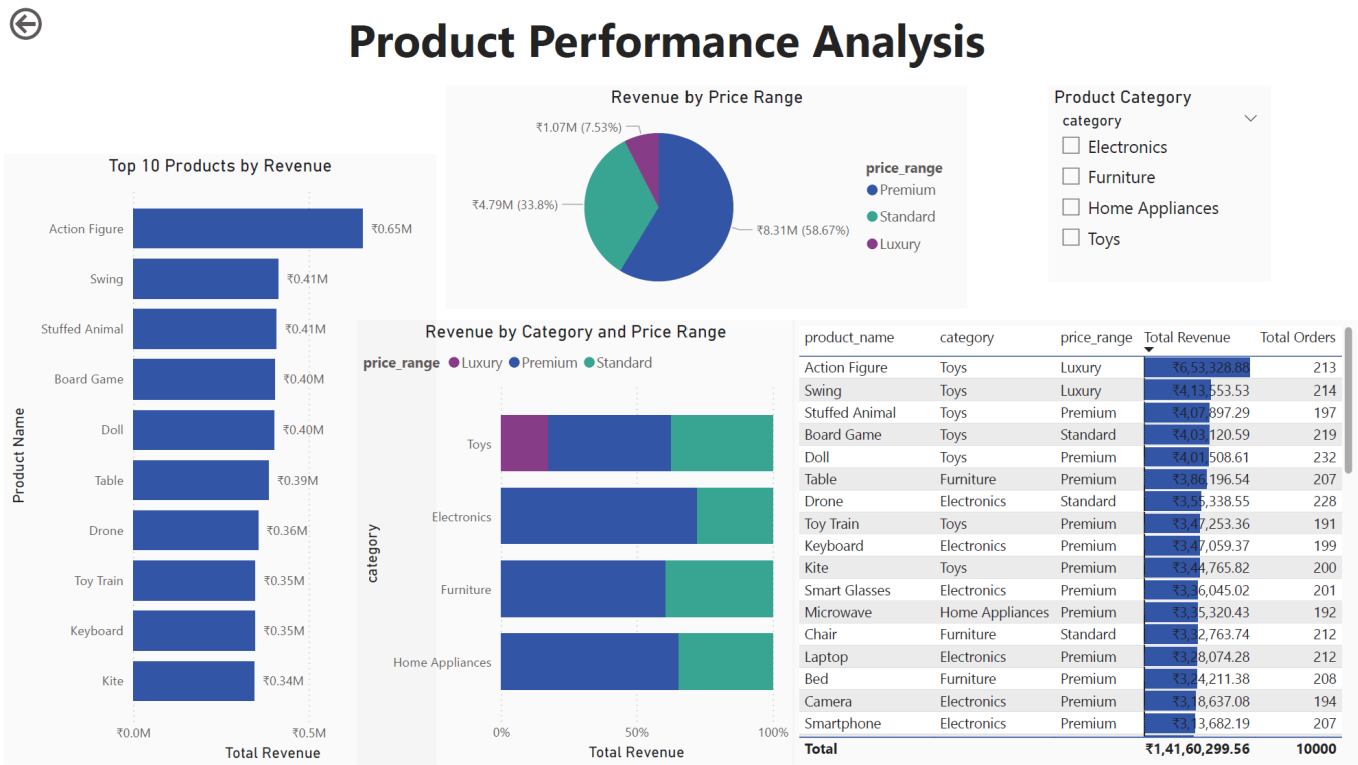
5.3 Page 3: Product Performance Analysis

This page answers:

Which products and price tiers are generating the most revenue, and how does performance distribute across categories?

It is also the destination page for Drill-through configured from Page 1 (Sales Overview)

> **NOTE:** The dashboard uses Indian Rupee (₹) currency formatting throughout. Currency was set in Power BI via Format → Data type → Fixed Decimal with ₹ prefix. The source data contains numeric sales amounts with no currency metadata , ₹ formatting was applied as a display convention.



product_name	category	price_range	Total Revenue	Total Orders
Action Figure	Toys	Luxury	₹6,53,328.88	213
Swing	Toys	Luxury	₹4,13,553.53	214
Stuffed Animal	Toys	Premium	₹4,07,897.29	197
Board Game	Toys	Standard	₹4,03,120.59	219
Doll	Toys	Premium	₹4,01,508.61	232
Table	Furniture	Premium	₹3,86,196.54	207
Drone	Electronics	Standard	₹3,55,338.55	228
Toy Train	Toys	Premium	₹3,47,253.36	191
Keyboard	Electronics	Premium	₹3,47,059.37	199
Kite	Toys	Premium	₹3,44,765.82	200
Smart Glasses	Electronics	Premium	₹3,36,045.02	201
Microwave	Home Appliances	Premium	₹3,35,320.43	192
Chair	Furniture	Standard	₹3,32,763.74	212
Laptop	Electronics	Premium	₹3,28,074.28	212
Bed	Furniture	Premium	₹3,24,211.38	208
Camera	Electronics	Premium	₹3,18,637.08	194
Smartphone	Electronics	Premium	₹3,13,682.19	207
Total			**₹1,41,60,299.56**	**10000**

6. Verification & Final Outcome

The notebook also queries the live retail_dw database to verify that all tables are loaded correctly and all KPI values match the Power BI dashboard. This acts as a reconciliation step between the SQL layer and the BI layer, output examples are added.

```
KPI SUMMARY  (sourced from MySQL · vw_kpi_summary)
====================================================
Total Revenue            : ₹  14,160,299.56
Total Orders             :          10,000
Avg Transaction Value    : ₹       1,416.03
Unique Customers         :           1,000
Customer Lifetime Value  : ₹      14,160.30
YoY Sales Growth Rate    : N/A
```

Top 5 Products by Revenue (vw_product_performance):

	sales_rank	product_name	category	total_sales
0	1	Action Figure	Toys	653328.88
1	2	Swing	Toys	413553.53
2	3	Stuffed Animal	Toys	407897.29
3	4	Board Game	Toys	403120.59
4	5	Doll	Toys	401508.61

6.1 Deliverables Checklist

- **Python + SQL ETL Pipeline:** retail_final.ipynb, code cells covering multi-format extraction (CSV/JSON/MySQL), cleaning, feature engineering, staging, star schema DDL, ETL load, reporting views, and KPI verification.

- **Data Warehouse (retail_dw):** 4 tables (DIM_CUSTOMER, DIM_PRODUCT, DIM_DATE, FACT_SALES) with enforced foreign keys, indexes, staging layer, and 7 analytical views.
- **Power BI Dashboard:** 3 pages: Retail Sales Overview, Customer Insights & Demographics, Product Performance Analysis , with 8 DAX measures, 3 relationships, interactive slicers and cross-page navigation.
- **KPIs Delivered:** Total Revenue, Total Orders, Avg Transaction Value, Unique Customers, Customer LTV, MoM Growth, YoY Growth (N/A - single year), Revenue by Category, Revenue by Price Range, Top Customers, Regional Performance.

Forward implementation plans-

- **Data Gap: Product Return Rate:** No returns data in source files. Metric is out of scope. *Forward implementation plan* if this data had been available, the pipeline would have included a returns dimension table, a return_rate_pct column in FACT_SALES (returned_units / total_units_sold × 100), and a dedicated returns analysis visual on the Product Performance page. A DAX measure Return Rate % = DIVIDE(SUM(FACT_SALES[returned_units]), SUM(FACT_SALES[quantity])) * 100 would have been created to track this by category and region.

- **Data Gap: Cost of Goods Sold (COGS):** No supplier cost data in source files. Gross Margin and Margin % are therefore also out of scope. *Forward implementation plan* if COGS data was available, it would have been integrated at the product level into DIM_PRODUCT (unit_cost column) and used in FACT_SALES to derive gross_margin_amount = sales_amount − (quantity × unit_cost). Key forward measures would include: Gross Margin % = DIVIDE([Total Revenue] − [Total COGS], [Total Revenue]) * 100 and Gross Margin by Category, enabling the business to identify whether high-revenue categories are also high-margin, which is a fundamentally different and more actionable insight.

Thank You